FinePar: A Fine-Grained Data Partitioning Framework for Cloud-Edge-Device Architectures

Shuangshuang Cui, *Student Member, IEEE*, Hongzhi Wang, *Senior Member, IEEE*, Jinghan Lin, Xiaoou Ding, and Donghua Yang

***Abstract*—The Cloud-Edge-Device (CED) architecture has emerged as a new framework for real-time data processing in the Internet of Things (IoT) era. However, the edge and device face significant resource constraints that prevent them from storing or processing full datasets. Effective data partition across CED architectures is therefore critical for supporting real-time decision-making. However, existing static and coarse-grained dynamic methods fail to adapt to changing workloads and to meet real-time processing demands. To address this issue, we propose FinePar, a fine-grained dynamic data partitioning framework based on DRL, coupled with an efficient data allocation strategy. FinePar combines horizontal and vertical partition to optimize data partition across CED architectures to reduce data transfer volume and shorten execution time. We use DRL to adjust data partitioning strategies in real time based on task demands and resource states. To achieve end-to-end optimization, we design an efficient data allocation strategy. We verified the effectiveness of FinePar through extensive experiments. Experimental results show that FinePar can reduce the edge side latency by 80% under resource constraints and dynamically adapts to workload changes.**

***Index Terms*—Cloud-edge-device architecture, data partition, storage optimization, deep reinforcement learning.**

I. Introduction

With the development of Internet of Things (IoT) [1] and edge computing, the Cloud-Edge-Device (CED) collaborative architecture [2], [3], [4] has become a core foundation for large-scale data processing and intelligent decision-making. Many scenarios require efficient processing of OLAP workloads to support real-time analytics [5]. However, due to resource limitations, such as storage space, computing power, and bandwidth, the edge and device nodes are unable to handle complete datasets and often fail to meet real-time processing demands, especially for dynamic OLAP queries where data access patterns change frequently. Although the cloud has powerful computing and storage capabilities, high latency and bandwidth limitations among the cloud, edge, and device make it impractical to offload all edge tasks to the cloud, especially for real-time applications. Therefore, how to achieve efficient data partition to support real-time decision-making and handle OLAP queries efficiently across the CED architecture has emerged as a key challenge.

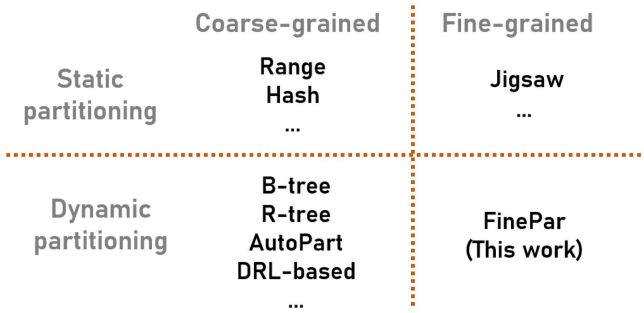


Fig. 1. The Data partitioning works.

In this context, data partitioning strategy is particularly crucial for resource-constrained edge and device nodes, as it determines how data can be distributed and processed efficiently across the CED. To ensure real-time processing, the edge and device nodes must dynamically decide which data to store locally and which should be offloaded to the cloud. Fig. 1 summarizes the existing partitioning works. However, traditional static partitioning methods often lack the flexibility and responsiveness required in such dynamic and heterogeneous environments.

Static partitioning methods typically determine partition rules during system initialization or data import and maintain them unchanged throughout system operation. Classical approaches such as range partitioning and hash partitioning divide data tuples horizontally based on specific attributes using predefined range or hash functions [6]. These coarse-grained partition methods are widely adopted in mainstream database systems including MySQL [9], Hive [10], and Spark [11]. They generally assume that computing and storage resources are balanced across nodes, which is often unrealistic in the resource-constrained CED environment. Fine-grained partitioning methods, such as Jigsaw [38], divide data along both the row and column dimensions. They perform well for fixed workloads but

Received 23 June 2025; revised 27 November 2025; accepted 15 January 2026. Date of publication 29 January 2026; date of current version 9 May 2026. This work was supported by the National Natural Science Foundation of China (NSFC) under Grant 62232005, Grant 62572151, in part by the Natural Science Foundation Project of Heilongjiang Province under Grant YQ2024F005, and in part by the CCF-Huawei Populus euphratica Innovation Research Funding under Grant CCF-Huawei DB2025004. Recommended for acceptance by S. Shang.*(Corresponding authors: Hongzhi Wang; Donghua Yang.)*

The authors are with the Faculty of Computing, Harbin Institute of Technology, Harbin 150001, China (e-mail: cuishuangs@stu.hit.edu.cn; wangzh@hit.edu.cn; linjinghan@stu.hit.edu.cn; dingxiaoou@hit.edu.cn; yang.dh@hit.edu.cn).

Digital Object Identifier 10.1109/TKDE.2026.3659213

1041-4347 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

are inefficient for dynamic ones due to high repartitioning costs. Nevertheless, traditional static partition methods often overlook resource constraints and workload changes, resulting in significant data transmission overhead in CED architectures.

To address these limitations, various dynamic partition methods have been proposed. These approaches adjust data placement at runtime based on system states and workload characteristics. Techniques such as B-tree, R-tree, and k-d tree [7], [8], [12], [13], [14], [15], [16] typically generate horizontal or vertical partitions based on data distribution. However, they often overlook the distribution of query workloads, limiting their effectiveness in dynamic CED architectures. Other heuristic dynamic methods, such as AutoPart [20], aim to recommend partition strategies but often rely on simplifying assumptions and struggle to handle complex workloads. Deep reinforcement learning (DRL)-based methods [21], [34], [37] have also been explored, but most focus on coarse-grained horizontal or vertical partitioning, which can result in redundant data scans and limited flexibility [42].

Although dynamic partition methods offer advantages over static ones, they still face significant challenges when directly applied to the CED architecture.

Fine-grained strategies enable systems to meet real-time requirements: The edge and device nodes in CED are typically constrained by limited computational power, memory, and bandwidth. As a result, they are unable to handle large data partitions generated by coarse-grained methods. In addition, coarse-grained partitioning often leads to redundant data storage across nodes and repeated data scans [42]. These problems increase system latency and waste resources. They are especially severe in edge scenarios that require real-time response. Therefore, there is a critical need for fine-grained partitioning approaches to improve efficiency and meet the real-time requirements in the CED architecture.

A feedback mechanism enables adaptive updates to partitioning strategies: Most existing dynamic partitioning methods rely on rule-based [17] or heuristic strategies [20], which lack the ability to continuously learn and optimize based on system feedback. In the resource-constrained and dynamically changing environment of the CED architecture, partitioning strategies must adapt to variations in workloads and resource availability. DRL offers a promising solution by enabling the system to learn effective partitioning strategies through interaction with the environment [22], [24], [25], [26], [27], [28], [29], [30], [31], [32]. However, few existing approaches fully exploit DRL in the context of CED's complexity, due to the challenges such as high-dimensional state spaces and heterogeneous resource constraints [33].

Partition allocation across CED layers is essential for end-to-end optimization: In CED architectures, designing an effective partitioning strategy alone is insufficient. Due to resource constraints and dynamic workload variations at the edge and device levels, each partition must be allocated based on real-time resource availability. This allocation has a direct impact on overall system performance. Focusing solely on partitioning logic while ignoring deployment locations prevents end-to-end optimization. Therefore, partition allocation is the key to achieve efficient and flexible data partitioning in the CED architecture.

In summary, existing methods cannot meet the needs of CED architectures. They rely on coarse-grained partitioning, lack adaptivity, and ignore partition allocation across layers. These limitations make it hard to support real-time tasks under limited and changing resources. In particular, rule-based and heuristic methods fail to learn from changing workloads and system states. This limitation motivates the use of DRL, which can learn from environment feedback and improve partition strategies over time [22], [24], [25], [26], [27], [28], [29], [30], [31], [32]. DRL is well-suited to capture the complex and dynamic nature of CED architectures, making it a promising solution for generating adaptive and fine-grained partition strategies.

To address these challenges with DRL, we propose FinePar, a fine-grained dynamic data partitioning framework, coupled with an efficient data allocation strategy for CED architectures. FinePar utilizes the adaptive decision-making ability of DRL to dynamically partition and allocate data efficiently across CED architectures, adapting to changes in resource availability and workload. To the best of our knowledge, this is the first work that uses DRL to recommend fine-grained data partitioning strategy for CED.

Firstly, to address the resource limitations of edge and device and the real-time task requirements, we design a fine-grained hybrid partitioning method. It combines horizontal and vertical partitioning to optimize data partition across CED for improving resource utilization and reducing latency.

Second, to enable dynamic adaptation, we use DRL to adjust data partitioning strategies in real time based on task demands and resource states. Unlike traditional heuristics that repeatedly make suboptimal decisions due to the neglect of experiences, we adopt DRL that learns from experience to progressively refine partitioning strategies.

The last but not least, in order to achieve end-to-end optimization, we formulate a temperature function to evaluate the each partition temperature and employ a storage allocation strategy to optimize data placement across the cloud, edge, and device layers. It further improves resource utilization and system performance in real-time. Consequently, FinePar ensures effective data allocation and enhances the performance of real-time decision tasks.

The contributions are listed as follows:

1) We propose FinePar, a fine-grained dynamic data partitioning framework based on DRL, coupled with an efficient data allocation strategy. To the best of our knowledge, this is the first work that uses DRL to recommend fine-grained data partitioning for CED architectures.
2) We formally define and prove the NP-hardness of the fine-grained hybrid data partitioning problem in CED architectures, providing a theoretical foundation for addressing the resource limitations of edge and device nodes and the real-time task requirements.
3) We propose a DRL-based method to train an automatic hybrid partitioning advisor. This advisor recommends hybrid partitioning strategy based on data and workload of edge and device, dynamically optimizing the data partition across CED architectures.

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

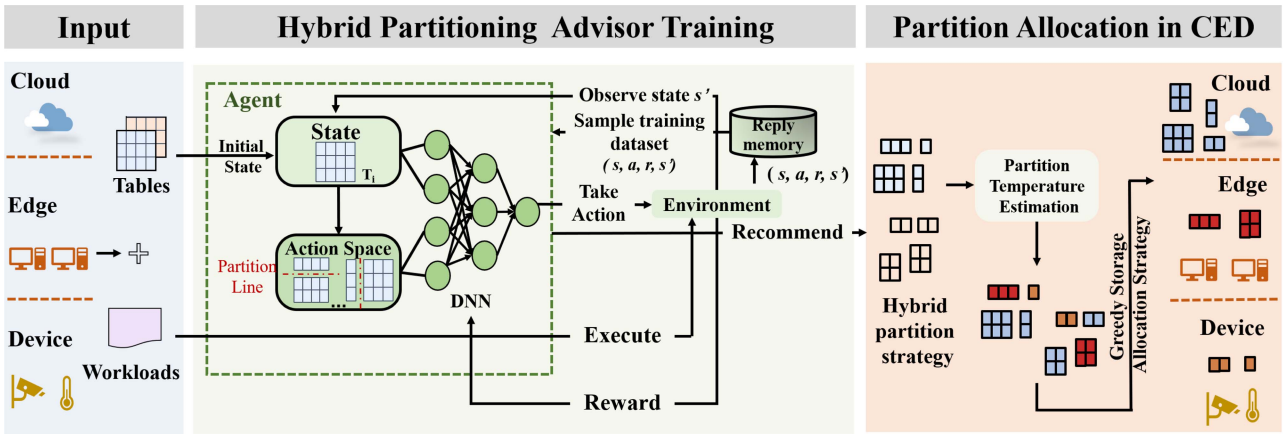


Fig. 2. An overview of FinePar's main stages: 1) the *Hybrid Partitioning Advisor Training* uses DRL to learn effective fine-grained hybrid partitioning strategies based on data and workload features as discussed in Section III, 2) the *Partition Allocation in CED* assigns fine-grained partitions to different layers based on partition temperature and system resource conditions as discussed in Section IV.

4) We perform partition allocation in CED architectures. We formulate a temperature function to estimate the each fine-grained partition temperature and employ a storage allocation strategy to optimize data placement across CED architectures. It can further improve resource utilization and system performance in real-time.
5) To clarify the effectiveness of our methods, we evaluate FinePar in three popular benchmarks, TPC-DS [39], TPC-H [40], and JOB [41]. Extensive experimental results show that FinePar can reduce the edge side latency by 80% under resource constraints and dynamically adjust the partitioning strategy based on the workload.

The rest of the paper is organized as follows: Section II provides an overview of FinePar; We introduce the hybrid partitioning advisor training in Section III; Section IV introduces the partition allocation in CED; Section V conducts experiments and analyzes the experimental results; The last part concludes and discusses the future work.

II. FinePar Overview

To achieve the efficient and flexible data partitioning in the CED architecture, we propose FinePar, a fine-grained dynamic data partitioning framework based on DRL, coupled with an efficient data allocation strategy. The architecture of FinePar is illustrated in Fig. 2. The main stages are: (1) the *Hybrid Partitioning Advisor Training*, which uses DRL to learn effective fine-grained hybrid partitioning strategies based on data and workload features. (2) the *Partition Allocation in CED*, which assigns fine-grained partitions to different layers based on partition temperature and system resource conditions.

System setting: The FinePar is designed for a collaborative CED environment that manages a distributed relational database. The input to the system consists of a set of relational tables and OLAP workloads, where each query performs analytical operations such as joins and aggregations on these tables. Each layer stores only a subset of data according to its storage capacity and computational capability. The optimization goal of FinePar is to determine how to partition and place data across devices to minimize overall query latency and data transfer cost, while satisfying the capacity constraint of each layer. Because edge and device nodes are resource-limited, they can only store frequently accessed or latency-sensitive data, whereas the cloud maintains cold data for reliability.

When a query requires data distributed across multiple layers, FinePar employs a coordinated query execution mechanism. The cloud node as the global coordinator and maintains metadata about the location of each data partition. Upon receiving a query, the coordinator identifies the involved partitions and determines the execution plan based on data locality and device resources. If most required data reside at an edge node, the query is executed locally, and only the missing partitions are fetched from the cloud. To remain within storage limits, FinePar supports adaptive replacement, where infrequently accessed partitions at the edge can be replaced with the newly fetched data. For cross-layer queries, the system performs distributed execution. Sub-queries are pushed down to corresponding devices for local computation, and upload the intermediate results to the cloud. This design ensures low latency, reduced data movement, and efficient resource utilization under capacity constraints.

Partitioning Workflow: FinePar consists of two main stages to enable adaptive fine-grained data partitioning in CED architectures. In the *Hybrid Partitioning Advisor Training* stage, we use DRL to learn effective hybrid partitioning strategies based on data and workload characteristics. In the *Partition Allocation in CED* stage, we assign fine-grained data partitions to the cloud, edge, or device layer according to partition temperature and real-time system resource conditions. Together, these stages allow FinePar to provide flexible and efficient partitioning strategies tailored to the requirements of CED architectures.

Hybrid Partitioning Advisor Training: This stage aims to learn effective fine-grained hybrid partitioning strategies through DRL. The key challenges in this stage include modeling the partitioning problem as a Markov Decision Process (MDP), designing an expressive state representation, defining an effective action space that captures both horizontal and vertical operations, and constructing a reward function that balances partitioning performance and resource efficiency. We address

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

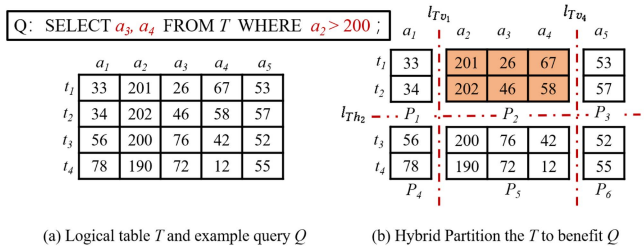
these challenges by training Deep Q-Network (DQN) with experience replay to approximate the optimal policy. Given data tables and workload characteristics in CED, the advisor encodes them into a formal state representation and defines an appropriate action space for partitioning decisions. Then, we trained deep neural network (DNN) to approximate the Q-function, guided by a carefully designed reward function and an experience replay mechanism. Through iterative learning, the advisor can recommend partitioning strategies that optimize data placement under dynamic resource and workload conditions. The modeling of the hybrid partitioning problem and the formulation of the DRL training environment are detailed in Section III.

Partition Allocation in CED: This stage is responsible for allocating the fine-grained hybrid partitioning strategies recommended by the advisor within the CED architecture. The major challenge of this stage lies in achieving lightweight decision-making without additional learning overhead, ensuring efficient data placement in resource-constrained and dynamic environments. To address this, we introduce a partition temperature function that guides partition allocation based on their workload characteristics. Given the hybrid partitioning, the system analyzes the access patterns of the workloads and calculates the temperature of each partition using a partition temperature function. This function computes the partition temperature based on the access frequency, partition size, and latency sensitivity, allowing the system to determine which partitions require low-latency access and which can tolerate longer retrieval times. Based on the partition temperatures and resource constraints of each layer, partitions are allocated to the appropriate storage layers to balance latency and storage efficiency. The detailed implementation of this allocation mechanism, including the partition temperature function, is described in Section IV.

Adapt to changing workload: To adapt to continuously changing workloads, FinePar employs a dynamic partitioning strategy based on DRL. The DRL agent continuously observes table access patterns, query latencies, and data access frequencies, and dynamically generates partition adjustment recommendations to balance storage distribution and query efficiency across cloud, edge, and end devices. Unlike static partitioning approaches, FinePar evaluates the utility of each partition and adjusts partition placement periodically based on observed workload changes. This enables the system to maintain high query performance and resource utilization under evolving access patterns. Achieving fully real-time adaptive partitioning is part of our future work. The mechanism requires no manual intervention, allowing FinePar to efficiently handle typical dynamic OLAP workloads.

III. Hybrid Partitioning Advisor Training

As discussed in Section I, efficient data partitioning for real-time tasks is the key challenge in CED architectures. Coarse-grained methods work in static environments but fail in resource-constrained and dynamic edge settings. Fine-grained hybrid partitioning has emerged as a crucial research direction. In the following, we first define the fine-grained hybrid partitioning problem and prove its NP-hardness in Section III-A. Then, we formulate our problem as a MDP in Section III-B and present the details of our DRL-based solution in Section III-C.



(a) Logical table T and example query Q (b) Hybrid Partition the T to benefit Q

Fig. 3. An example of hybrid partition for table T.

A. Hybrid Partitioning

Definition 1 (Partition Line). A partition line represents the location for horizontally or vertically partition table. Consider a table T with tuples $\{t_1, t_2, \ldots, t_m\}$ and attributes $\{a_1, a_2, \ldots, a_n\}$, divide m tuples heuristically into b blocks, the horizontal partition line is denoted by $l_{Th} = \{l_{Th_i} | 0 < i \leq b-1\}$, and the vertical partition line is denoted by $l_{Tv} = \{l_{Tv_j} | 0 < j \leq n-1\}$. The set of partition lines for T is denoted by $\mathbb{L}_T = \{l_T | l_T \in l_{Th} \cup l_{Tv}\}$.

For example, Fig. 3(a) shows the table T, $l_{Th} = \{l_{Th_1}, l_{Th_2}, l_{Th_3}\}$ and $l_{Tv} = \{l_{Tv_1}, l_{Tv_2}, l_{Tv_3}, l_{Tv_4}\}$. We can get Fig. 3(b) when $l_{Th} = \{l_{Th_2}\}$, $l_{Tv} = \{l_{Tv_1}, l_{Tv_4}\}$.

Definition 2 (Hybrid Partition Problem in CED). Given tables $\{T_1, T_2, \ldots, T_N\}$ with partition lines $\{\mathbb{L}_{T_1}, \mathbb{L}_{T_2}, \ldots, \mathbb{L}_{T_N}\}$, the workload $\{q_1, q_2, \ldots, q_M\}$, the predefined threshold of workload performance W in CED and the maximum number of partition lines C. $\mathbb{L}_{T_N}$ denotes all the partition lines in the table T_N, in which 1 indicates the partition line will be used, while 0 means no such partition line. The goal of Hybrid Partition Problem is to find $\mathbb{L}_{T_a}$ for each table T_a to vertical and horizontal partition them, such that the performance of the given workload using $\mathbb{L}_{T_a}$ not less than W, and the size of $\mathbb{L}_{T_a}$ is not more than C.

Theorem 1. The hybrid partitioning problem in CED is NP-hard.

Proof. The problem is clearly in NP, since the performance of the given workload (e.g. Runtime) using $\mathbb{L}_{T_a}$ could be checked in polynomial time.

To show the NP-completeness, we can build a reduction from the Knapsack problem in polynomial time. The Knapsack problem is one of the Karp's 21 NP-complete problems [43]. The definition of 0-1 Knapsack problem is as follows [44].

There are n kinds of items denoted by M_1, M_2,..., M_n, and let $M = \{M_1, M_2, \ldots, M_n\}$. Each kind of item j has a weight S_j and a value V_j. It is assumed that all weights and values are nonnegative. The maximum weight can be carried in the bag is K, and E is defined as the predefined threshold of the total value of items, where K and E are both nonnegative values. The number X_j of copies of each kind of item is restricted to zero or one. The object of 0-1 Knapsack problem is to find a subset of items, equivalent to assigning value to each X_j), subjecting

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

to the following two conditions:

$$\sum_{j=1}^{n} S_j X_j \leq K$$

$$\sum_{j=1}^{n} V_j X_j \geq E, X_j \in \{0,1\}$$

We build a reduction from the Knapsack problem including three steps as follows.

Step 1: Instance Construction.

We first construct a specific set $\mathbb{L}_{T_a} = \{L_{T_1}, L_{T_2}, \ldots, L_{T_n}\}$ in the hybrid partition problem. The set $\mathbb{L}_{T_a}$ is consist of n partition lines. The number of each partition line j is S_j. The performance of the given workload using partition line j is V_j. Let the predefined threshold C be equal to K, and let the predefined threshold W be equal to E.

Step 2: if the solution of 0-1 Knapsack problem exists, then the solution of the hybrid partition problem also exists.

If the solution of 0-1 Knapsack problem exists, i.e., $\exists M' \subseteq M$, such that

$$\sum_{M_j \in M'} S_j \leq K$$

$$\sum_{M_j \in M'} V_j \geq E, 1 \leq j \leq n$$

Then $\exists \mathbb{L}'_{T_a} \subseteq \mathbb{L}_{T_a}$, such that

$$\sum_{L_{T_j} \in \mathbb{L}'_{T_a}} S_j \leq C$$

$$\sum_{L_{T_j} \in \mathbb{L}'_{T_a}} V_j \geq W, 1 \leq j \leq n$$

Therefore, there exists a specific set $\mathbb{L}_{T_a}$ for each table T_a so that the performance of the given workload, e.g., Runtime, using $\mathbb{L}_{T_a}$ not less than W and the size of $\mathbb{L}_{T_a}$ is not more than C. Consequently, the solution of the hybrid partition problem instance exists.

Step 3: if the solution of the hybrid partition problem exists, then the solution of 0-1 Knapsack problem also exists.

If the solution of the hybrid partition problem exists, i.e., there exists a specific set $\mathbb{L}_{T_a}$ for each table T_a so that the performance of the given workload (e.g. Runtime) using $\mathbb{L}_{T_a}$ not less than W, and the size of $\mathbb{L}_{T_a}$ is no more than C. Let PL denote the set of partition lines involved in the $\mathbb{L}_{T_a}$. Let the function $F(PL)$ denote the total performance of the given workload only using partition lines in PL, and let N_{PL} denote the number of partition lines in PL. Then we can get that

$$N_{PL} \leq C$$

$$F(PL) \geq W$$

We divide PL into two subsets: PL_1 and PL_2, subject to $PL = PL_1 \cup PL_2$, $PL_1 \cap PL_2 = \emptyset$. PL_1 and PL_2 are defined by $PL_1 = \bigcup L_{T_j}, L_{T_j} \subseteq PL, 1 \leq j \leq n$, $PL_2 = PL - PL_1$. As PL_2 cannot be used to partition any table, we can see that $f(PL_2) = 0$. Therefore, $f(PL) = f(PL_1) + f(PL_2) = f(PL_1) \geq W$. On the other hand, as PL_1 is the subset of PL, $N_{PL_1} \leq N_{PL} \leq C$. Therefore, we can find the subset of items M', such that

$$\sum_{M_j \in M'} S_j = N_{PL_1} \leq K$$

$$\sum_{M_j \in M'} V_j = F(PL_1) \geq E$$

Consequently, the solution of 0-1 Knapsack problem exists.

Since the Knapsack problem is NP-complete, and we have shown a polynomial-time reduction from the Knapsack problem to the hybrid partition problem in CED, the hybrid partition problem in CED is NP-hard. ■

Based on the three-step proof above, we conclude that the hybrid partition problem in CED is an NP-hard problem.

We have proved the hybrid data partition problem in CED is NP-hard, since the traditional heuristics such as greedy methods cannot be adapted to the cost observed during search, there should be a trade-off between exploitation and exploration when choosing a partitioning strategy. We model the process of data partition as a MDP and solve the problem with DRL efficiently.

B. Hybrid Partition as DRL

To facilitate understanding of the DRL-based design of FinePar, we first introduce the fundamental concepts of reinforcement learning. Specifically, the learning process can be modeled as a MDP, where the agent learns an optimal policy through interaction with the environment. We further review the classical Q-Learning algorithm, which serves as the foundation for the DRL formulation adopted in FinePar.

The Bellman's Principle of Optimality and the characterization of dynamic programming are among the most significant achievements in computing [47]. They are intricately linked to a subset of stochastic processes known as MDPs. MDPs serve as a formal framework for a range of issues. In MDPs, an agent aims to optimize a predefined objective, such as enhancing performance or precision, by making a sequence of choices. Each choice is contingent on the current state and typically results in a transition to a new state. The term "Markovian" alludes to the fact that the current state of the system fully dictates its future development.

Reinforcement learning (RL) is a class of stochastic optimization techniques for MDPs. In RL, an agent's interaction with its environment involves making choices. Specifically, at each discrete time step t, the agent observes the current state s_t and selects an action a from a set of available actions A. This choice leads to a transition to a new state s_{t+1} and the receipt of a reward r_t. The way the agent selects actions based on the current state is known as the policy π. The agent's ultimate goal is to maximize the total rewards over time. However, a greedy policy, which opts for the action with the highest immediate reward, may not be the most effective strategy. Instead, the agent may choose actions that will yield higher rewards in the future. Therefore, when making decisions, the agent should always consider long-term rewards.

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

Q-learning is a solution to solve RL problem. In Q-learning, when we pick action a at state s, the expected discounted future rewards are approximated with Q-function as follows: $Q(s,a) = \mathbb{E}(\sum_{t=0}^{\infty} r_t(s_t,a_t)\gamma^t | s_0 = s, a_0 = a)$. To cope with the uncertainty of the future state, rewards are discounted by a discount factor $\gamma < 1$. The Q-function is learned during training. There are different ways to realize the Q-function. In DRL, a neural network $Q_\theta(s,a)$ with weights θ is used for approximation. After observing a state s_t and an action a_t, the network is updated by Stochastic Gradient Descent (SGD) utilizing the corresponding immediate reward r_t and future state s_{t+1}. The expected discounted future rewards should be the immediate reward r_t together with the maximum expected discounted future rewards when selecting the best action a in the next step $t+1$ discounted by γ. The squared error loss is $(r_t + \gamma \arg\max_{a \in A} Q(s_{t+1}, a_t) - Q(s_t, a_t))^2$.

To formulate the hybrid partition problem as a DRL problem, we model tables as states and all possible partition operations on the table as actions. The reward is the performance gain for a given workload.

States: Since we focus on hybrid partition strategies, it is important to represent the partition of tables. As described in the previous section, hybrid partition mainly consists of horizontal partition for certain tuples and vertical partition for certain attributes. Therefore, we use the one-hot encoding $s(T_i) = (l_{th1}, l_{th2}, \ldots, l_{th(b-1)}, l_{tv1}, l_{tv2}, \ldots, l_{tv(n-1)})$ to represent the partition of Table T_i. The $l_{th(b-1)}$ and $l_{tv(n-1)}$ encoded 0 or 1 to represent whether a partition line is used for horizontal or vertical partition. Q-learning is a simple off-policy algorithm for temporal difference learning, corresponding to a model-free Reinforcement Learning method [48] used when the agent initially only knows the set of possible states and actions.

Challenges: Specifically, in scenarios with massive amounts of data, a table may contain millions of tuples. There are two challenges with horizontal partition line encoding. (1) How to control the number of encodings to ensure scalability? (2) How to prevent the invalidation of the mapping from horizontal partition lines to tuples when tuples are deleted? To address the mentioned challenges, we have deployed *tuple blocks* as the granularity for horizontal partitioning.

Key idea: Since we typically handles large scale complex range queries in CED, FinePar firstly used frequent itemset mining techniques to determine the sorting keys to automatically sort the data. Then the sorted tuples are divided into 10 *tuple blocks*. In this way, the encoding representing the horizontal partition lines requires only 9 bits, addressing the scalability limitations caused by tuple level encoding. Additionally, it can also avoid the invalidation of the mapping from horizontal partition lines to tuples when tuples are deleted. The choice of the block number reflects a trade-off: fewer blocks yield coarser partitions but support faster and more stable DRL learning, while more blocks offer finer-grained control at the expense of a larger state space and potentially slower convergence. The impact also depends on table size. Smaller tables inherently produce finer partitions, whereas larger tables result in coarser ones. We chose 10 blocks as a balanced setting that provides sufficient partition granularity while keeping the DRL state space manageable, ensuring stable and efficient learning.

Moreover, the DRL agent's behavior is expected to remain stable under moderate variations in the block number (e.g., 8–12) because the underlying MDP structure is largely preserved. Specifically, state transition probabilities are minimally affected as the global tuple order and access patterns remain essentially unchanged, with only slight adjustments in tuple aggregation; the reward function does not change with the block number; and the action space retains its core structure. Here, the number of block boundaries, defined as the positions between tuple blocks where horizontal partition lines can be placed, is equal to the number of blocks minus one. Moderate changes in block number (e.g., $10 \rightarrow 8$ or 12) only slightly alter the number of available block boundaries, causing limited perturbation to the action space and allowing the DRL policy to remain robust. These factors together ensure that moderate block number changes do not significantly alter the DRL learning dynamics, supporting the robustness of our chosen parameter range.

Future work can further optimize or even dynamically adapt the block number. A promising direction is to develop adaptive block-sizing mechanisms that adjust the number and boundaries of tuple blocks according to table scale, workload characteristics, or the DRL agent's learning dynamics. Such a design would improve generality across heterogeneous datasets and reduce reliance on manually preset hyperparameters, offering a pathway toward a more flexible and robust partition advisor.

Actions: we support two types of actions: (1) horizontal partition by the partition line when $\mathbb{L}_T = \{l_T | l_T \in l_{Th}\}$. (2) vertical partition by the partition line when $\mathbb{L}_T = \{l_T | l_T \in l_{Tv}\}$. Similar to state encoding, we still use one-hot encoding actions.

For instance, the table T is hybrid partitioned in Fig. 4, the initial state vector is $(l_{th1}, l_{th2}, l_{th3}, l_{tv1}, l_{tv2}, l_{tv3}, l_{tv4}) = (0,0,0,0,0,0,0)$. When it is horizontally partitioned by l_{th2}, its state vector is $(l_{th1}, l_{th2}, l_{th3}, l_{tv1}, l_{tv2}, l_{tv3}, l_{tv4}) = (0,1,0,0,0,0,0)$. When it is vertically partitioned by l_{tv1}, its state vector is $(l_{th1}, l_{th2}, l_{th3}, l_{tv1}, l_{tv2}, l_{tv3}, l_{tv4}) = (0,1,0,1,0,0,0)$. Finally, it is vertically partitioned by l_{tv4}, and its terminal state vector is $(l_{th1}, l_{th2}, l_{th3}, l_{tv1}, l_{tv2}, l_{tv3}, l_{tv4}) = (0,1,0,1,0,0,1)$.

The state s and an action a are then used as input to the neural network to predict the Q-value $Q(s,a)$.

Reward: The overall goal of the hybrid partition advisor is to find a suitable partition strategy for the workload that minimizes the sum of the partition block sizes accessed by the query. Therefore, we set the reward function to learn a DQN agent as follows.

$$r = -1 + \frac{last_cost}{cost} \tag{1}$$

where $last_cost$ is the cost of the last partition policy accessed by all queries in the workload, and $cost$ is the cost of the partition policy accessed by all queries in the workload.

C. Training Procedure

After modeling the hybrid partition problem as a DRL task with defined states, actions, and reward, the next step is to train the agent to learn an effective partition policy. Traditional Q-learning performs poorly when the state and action space is large, since searching for the appropriate state in a large table is a

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

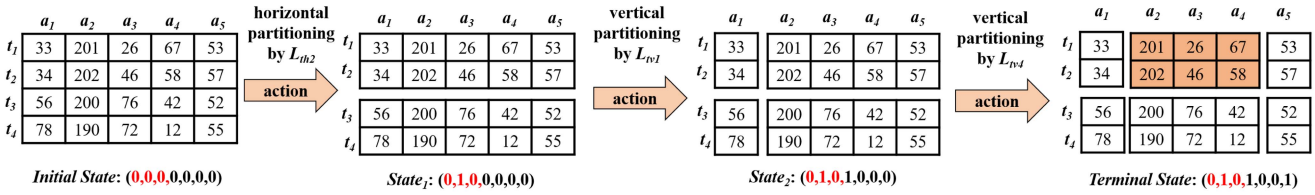


Fig. 4. An example of state representation.

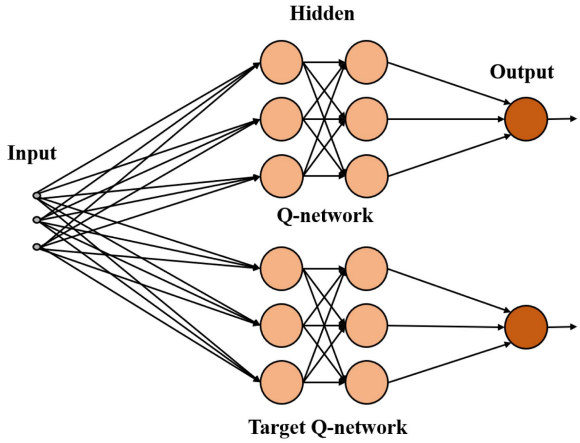


Fig. 5. The architecture of neural networks.

time-consuming task. DQN can solve the above problems well. In the following, we discuss the DRL-based training process in detail. DRL aims to maximize cumulative rewards when taking actions in a search space captured by MDP. Essentially, there is a trade-off between exploitation and exploration of what it learned.

We constructed two same neural networks. The architecture of each network is shown in Fig. 5. In order to improve the non-linear mapping ability of the network, we added two hidden layers. Even though a deeper network can achieve higher accuracy, we designed a two layers fully connected layer hidden network for efficiency issues.

During network training, the DQN chooses whether to partition the data horizontally or vertically in the current state. We set each training episode to a series of steps. We propose Hybrid Partitioning Training Algorithm to train the hybrid partition model. As we can see in Algorithm 1, given a table T, workload W, replay memory D, batch size bs, random probability ϵ, learning rate α, depreciation rate γ, at the beginning, we initialize Q-network Q with random weights θ and the target Q-network Q' with random weights θ'. Training consists of many episodes in Lines 3-13. During the training process, RL's parameter θ is continuously adjusted to adapt to the environment to achieve better performance. Finally, we obtain the trained parameters θ'.

At the beginning of each episode, in Line 4, the state is initialized, and in each episode. Lines 5-13 are performed multiple times. At the beginning of each step, the network RL selects the action according to the settings in Line 6. Typically, the exploration is performed by selecting a random action with probability ϵ. This probability decreases over time by multiplication with a factor called *epsilon decay*. After executing the action in the environment in Line 7, and according to the reward function in

Algorithm 1. Hybrid Partitioning Training Algorithm.

Input: Table T, Wokload W, Replay memory D, Batch size bs, Random probability ϵ, Learning rate α, Depreciation rate γ
Output: Trained Model parameters θ'

1 Initialize Q-network Q with random weights θ;
2 Initialize target Q-network Q' with random weights θ';
3 **for** *each episode* **do**
4 | initialize state
5 **for** *each step in episodes* **do**
6 | With probability ϵ select $a_t = argmax_a Q_\theta(s_{t+1}, a)$, otherwise random action a_t ;
7 | Execute action a_t ;
8 | Compute reward with $r = -1 + last_access_num/access_num$;
9 | Store transition (s_t, a_t, r_t, s_{t+1}) in D;
10 | Sample $bs(s_i, a_i, r_i, s_{i+1})$ from D;
11 | Train Q-network with SGD and loss $\sum_{i=1}^{b}(r_i + \gamma \arg\max_{a \in A} Q'_\theta(s_{i+1}, a_i) - Q_\theta(s_i, a_i))^2$;
12 | Respect to the weights θ ;
13 | Every H steps reset target Q-network $\theta' = \theta$;
14 return

Line 8, the reward is obtained. Due to the need to train the neural network model, Line 9 stores the current state, the action, the reward, and the next state as transitions into the experience pool. After a few steps of training, Line 10 samples some transitions from the experience pool to update the network for more accurate decisions. Line 11 trains the Q-network using the SGD and the loss function. The parameter θ is updated to the target network after every H step. So far, one step has been completed.

After training all episodes, the reinforcement learning neural network is adjusted to the optimal state. And the models of different episodes are saved for easy loading and using again without retraining.

The time complexity of Algorithm 1 is $O(I\tau)$, where I is the total number of training episodes and τ is the total number of steps in each episode.

Through iterative learning, the advisor can recommend fine-grained partitioning strategies that optimize data placement under dynamic resource and workload conditions.

IV. Partition Allocation in CED

After the hybrid partitioning advisor generates fine-grained partitioning strategies, the next critical step is to determine how to allocate these partitions across the cloud, edge, and

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

device layers. This partition allocation process is essential for achieving end-to-end optimization in the CED architecture, as it directly affects data access latency, resource utilization, and overall system responsiveness. Partition allocation in the CED architecture is challenging due to the heterogeneous and limited resources at edge and device nodes, which cannot store all partitions locally. Additionally, dynamic workload and resource fluctuations, along with the lack of global system knowledge, demand adaptive and lightweight allocation strategies. In this section, we first introduce how to estimate partition temperature in Section IV-A. Then we describe the greedy storage allocation strategy in Section IV-B.

A. Partition Temperature Estimation

The partition allocation in the CED architecture introduces several challenges. First, the heterogeneous and limited resources of edge and device nodes, such as constrained memory, compute power, and bandwidth, which make it infeasible to store all partitions locally. Second, the dynamic variation of workloads and resource availability requires allocation strategies to be adaptive and lightweight, rather than static or manually configured. Third, the lack of global knowledge across distributed nodes further complicates the decision-making process.

To address these challenges, an effective allocation strategy must evaluate the importance of each partition, such as its access frequency or latency sensitivity. To this end, we define a partition temperature function to quantify the temperature of each partition. This unified metric clearly shows the importance of each partition in real-time scenarios. It helps make adaptive and efficient data placement decisions. The function is also efficient to compute and easy to extend. This makes it suitable for the resource-limited and dynamic CED environment.

Given a hybrid partitioning strategy $P = \{p_1, p_2, \ldots, p_n\}$, each p_i represents a fine-grained partition block. The features of each partition block include access frequency, partition size, and data access latency sensitivity, which refers to whether low-latency access is required. To guide the partition allocation process, we define a partition temperature function $H(p_i)$ for each partition p_i to quantify the temperature. The temperature function combines access frequency, partition size, and latency sensitivity into a single score:

$$H(p_i) = \mu_1 \cdot Norm(f_i) + \mu_2 \cdot Norm(w_i) - \mu_3 \cdot Norm(s_i) \tag{2}$$

Where f_i is the access frequency of partition P_i, w_i is the latency sensitivity, and s_i is the size of the partition. μ_1, μ_2, and μ_3 are weight parameters that control the relative importance of each factor. $Norm()$ is the normalization function that maps the input to the range $[0, 1]$.

The temperature function considers access frequency, latency sensitivity, and partition size, which are selected to reflect the operational characteristics of CED environments. Specifically, access frequency captures how often a partition is needed, indicating its importance for real-time tasks. Partition size is included to account for storage pressure, especially in edge and device nodes with limited memory. However, relying solely on access frequency and partition size cannot capture the temporal urgency of data access, which is crucial for real-time tasks in CED environments. Latency sensitivity characterizes the extent to which a task relies on timely data access. A higher sensitivity indicates that even small access delays to partition p_i can significantly affect the task's quality of service. This factor is essential in CED systems, where latency-critical tasks such as real-time monitoring and control need to be processed at nearby edge nodes, even when their access frequency remains moderate. By incorporating latency sensitivity, the temperature function can distinguish between frequently accessed data and latency-critical data, enabling more precise prioritization in partition placement. These features jointly provide a practical and balanced measure for guiding data placement.

It is worth noting that the design of the temperature function balances simplicity and effectiveness. The weights assigned to each feature can be adjusted or learned through system feedback. Furthermore, the temperature scores can be discretized to simplify subsequent decision-making rules. This flexibility makes the temperature function practical and easily extendable across diverse CED environments.

B. Greedy Storage Allocation Strategy

After designing the partition temperature function to measure the importance of each partition, there are still several challenges in assigning partitions to cloud, edge, and device layers based on their temperature scores. First, the CED has heterogeneous and limited resources in each layer. The edge and device layers especially have limited storage and cannot hold all important partitions. Second, workloads and resource availability change dynamically, so the allocation strategy must respond in real time and adjust accordingly. Third, nodes in the CED are distributed and lack a global view, so decisions must be made locally but still try to be optimal. Given these challenges, the allocation strategy must not only place data reasonably but also consider system performance and real-time requirements.

To address these challenges, we introduce a greedy storage allocation strategy that aims to assign partitions to cloud, edge, and device layers while respecting resource constraints and maximizing system efficiency.

Given the partition temperature rankings, the system assigns partitions to different storage layers based on available capacity in each layer. The storage capacities of the device and edge layers are S_{device} and S_{edge}. The greedy algorithm operates as follows:

Step 1. Sort all partitions P_i in descending order of their temperature scores $H(p_i)$.

Step 2. Assign partitions with the highest temperatures to the edge layer until S_{edge} is filled.

Step 3. Assign the next set of partitions to the device layer until S_{device} is filled.

Step 4. Any remaining partitions are placed in the Cloud layer.

The allocation decision $L(p_i)$ for each partition is formulated as:

$$L(p_i) = \begin{cases} \text{Edge}, & \text{if } \sum_{p_j \in \text{Edge}} \text{size}(p_j) + \text{size}(p_i) \leq S_{\text{edge}} \\ \text{Device}, & \text{if } \sum_{p_j \in \text{Device}} \text{size}(p_j) + \text{size}(p_i) \leq S_{\text{device}} \\ \text{Cloud}, & \text{otherwise} \end{cases} \tag{3}$$

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

The major advantage of the greedy algorithm is its low computational cost and simple implementation, which allows it to respond quickly in dynamic environments. Compared to complex global optimization methods, the greedy strategy does not need large-scale state space searches, making it suitable for CED systems with limited resources and real-time requirements. In addition, the intuitive rules of greedy allocation make the system easier to deploy and maintain, improving its practicality and scalability.

However, the greedy strategy also has some limitations. Since it makes decisions based only on the current state and local optimality, it may result in suboptimal global allocation. For example, assigning the hottest partitions first may leave no space for other important ones, forcing them to be stored in the cloud, which increases access latency. Even so, in CED environments, fast response and efficiency are usually more important than global optimality. Therefore, the greedy algorithm is still useful in practice. Future work can combine it with heuristic search or learning-based methods to improve partition allocation results.

With a partition allocation strategy, we can reasonably store hybrid partitions across each layer of the CED architecture. This placement aligns data with the resource characteristics of the cloud, edge, and device tiers. Frequently accessed and latency-sensitive data are placed on faster and closer nodes. Less critical data are offloaded to higher-capacity but distant layers. This reduces access latency and improves system responsiveness. It also balances storage load and avoids unnecessary data movement. As a result, the system can support real-time decision-making with better efficiency and resource utilization. Such allocation is especially important in dynamic CED environments, where workloads and available resources change over time.

V. Evaluation

In this section, we conduct extensive experiments to verify the performance of the proposed approaches.

A. Experimental Settings

We used PostgreSQL 15.4 as the relational DBMS system, Python 3.7 and PyTorch 1.13.1 for developing and testing FinePar in CED architecture, which consists of a device (windows system Intel(R) Core(TM) i7-8565 U CPU @ 1.80 GHz 1.99 GHz and 8 GB RAM), an edge server (4 RTX A6000 GPU, Intel(R) Xeon(R) Silver 4210R CPU, 48 GB memory and 504 GB RAM), and a cloud server (8 NVIDIA Tesla V100 GPU, Intel(R) Xeon(R) Gold 6133 CPU @ 2.50 GHz, 40 GB memory and 512 GB RAM). This setup allows us to clearly evaluate the effectiveness of the FinePar partitioning strategy while ensuring experiment reproducibility and analysis accuracy. The network bandwidth is 100 Mbps. We trained the hybrid partitioning advisor with an approximate Q-function consisting of two standard ReLU activation function layers of neural networks with 150 neurons. The hyperparameter settings for training are shown in Table I. We trained 500 episodes for three kinds of workloads. FinePar can use Python interface to interact PostgreSQL and extract the information of tables and workloads. It can give us hybrid partitioning strategies which consists of a set of partition actions.

TABLE I
Hyperparameter Settings for Training Hybrid Partition Advisor

Comments	Parameter	Value
Batch size	bs	32
Random probability	ϵ	0.5
Learning rate	α	0.04
Depreciation rate	γ	0.99
Max step size	t_{max}	10
Experience Replay Buffer Size	S	500
target replace iter	C	30

TABLE II
Partition Tables

Table_name	Rows	Columns
customer_address	383000	13
customer_demographics	1920800	9
household_demographics	7200	5
time_dim	86400	10
store	144	29
date_dim	73049	28
store_sales	144004764	23

Data and workloads: To conduct a comprehensive evaluation of FinePar, we utilized three workloads to analyze its performance.

1) TPC-DS: TPC-DS [39] comes with complex schema of 24 tables and 99 queries. We used the scaling factor SF = 30 and three TPC-DS query templates, Q13, Q14 and Q15 on the seven tables (as shown in Table II. Based on these three query templates, we generated a large number of queries.

2) TPC-H: TPC-H [40] is a standard industry database benchmark with 8 tables. we used the scaling factor SF = 30 and all TPC-H queries for evaluation.

3) JOB: Join order benchmark [41] is derived from the Internet Movie Data Base (IMDB). The dataset is 3.6 GB in size and consists of 21 relational tables. The largest table has 36 million rows. It contains 33 templates and 113 queries in total to evaluation.

Baseline methods: We compare FinePar with both static and dynamic partitioning baselines. It is particularly noted that Jigsaw [38], as a static fine-grained partitioning scheme, has limited adaptability to workload changes and is not suitable for CED architectures. Therefore, we did not include it as a baseline method.

- *SC-Origin:* This method is a **S**tatic **C**oarse-grained **Origin**al partitioning method, which refers to the Original table without any partitioning. No partition operation is performed on the table, and all data remains in a single structure. This method typically serves as a baseline for comparison, evaluating the effectiveness of other partitioning strategies.
- *SC-H:* This method is a **S**tatic **C**oarse-grained **H**orizontal partitioning method, which uses a heuristic approach based on a priori algorithm to extract frequent item sets, and subsequently identifies the columns corresponding to the predicate conditions as partitioning keys [18].
- *SC-V:* This method is a **S**tatic **C**oarse-grained **V**ertical partitioning method, which uses a heuristic approach to

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

TABLE III
RUN TIME REDUCTION (%) OF FinePar OVER BASELINE METHODS

Dataset	TPC-DS					TPC-H					JOB				
Query amount	SC-Origion	SC-H	SC-V	DC-DRL	Grep	SC-Origion	SC-H	SC-V	DC-DRL	Grep	SC-Origion	SC-H	SC-V	DC-DRL	Grep
20	44.89%	28.31%	42.12%	28.46%	21.30%	98.49%	94.14%	94.77%	92.95%	88.87%	97.77%	92.48%	93.50%	91.76%	87.64%
50	45.28%	27.24%	42.45%	27.63%	20.39%	95.36%	87.05%	86.59%	85.64%	77.31%	97.14%	90.30%	92.35%	89.39%	84.08%
100	40.17%	33.66%	37.82%	33.68%	27.05%	96.55%	89.05%	90.97%	87.06%	79.56%	97.32%	91.35%	92.51%	90.56%	85.84%
200	45.09%	27.28%	42.72%	28.41%	21.25%	96.93%	90.22%	91.75%	88.50%	81.83%	97.17%	90.83%	92.32%	90.04%	85.07%
500	44.21%	29.87%	41.83%	30.14%	23.16%	96.52%	89.32%	90.52%	87.51%	80.27%	97.76%	92.38%	93.93%	91.51%	87.27%
1000	41.99%	29.83%	43.51%	29.83%	22.82%	96.32%	88.88%	90.12%	86.95%	79.38%	97.76%	91.85%	93.51%	90.96%	86.44%
Average	**43.60%**	**29.37%**	**41.74%**	**29.69%**	**22.66%**	**96.69%**	**89.78%**	**90.78%**	**88.10%**	**81.20%**	**97.49%**	**91.53%**	**93.02%**	**90.70%**	**86.06%**

generate an affinity matrix between attributes based on the workload, and clusters them according to the affinity between attributes [19].

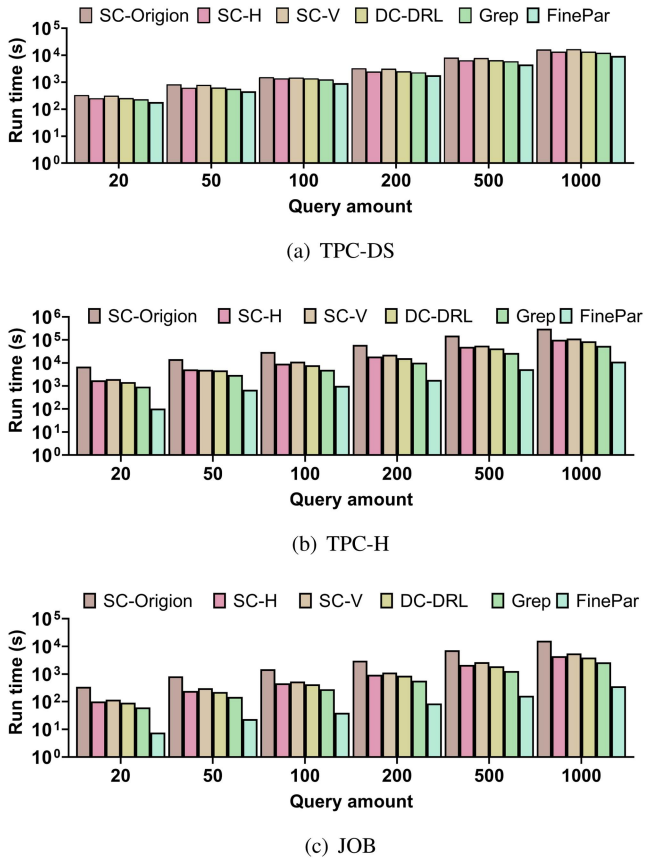
- *DC-DRL:* This method is a **D**ynamic **C**oarse-grained **DRL**-based partitioning method, which is a learned horizontal partition advisor based on DRL for OLAP-style workloads by Hilprecht et al. [21]. It supports horizontal hash partition. It also finds the partition key by DRL and horizontally partitions the table.
- *Grep:* This method is a dynamic coarse-grained partitioning method, which represents data relationships as graphs and leverages graph learning to optimize data distribution across nodes, thereby reducing inter-node communication and enhancing query performance [23].

Performance evaluation metrics: The metrics used in the experiment are Run time, Data transfer volume, and Scan size.

1) Run time: It measures the total time consumption (including edge hit time and data transmission time) of the response after the edge initiates a query.

2) Data transfer volume: It measures the total amount of data transmitted across different layers (e.g., from cloud to edge or device) during query execution, reflecting the system's communication overhead.

3) Scan size: It measures the size of the accessed partition block.



(a) TPC-DS

(b) TPC-H

(c) JOB

Fig. 6. Comparison of run time: FinePar vs. Baselines.

B. End-to-End Evaluation of FinePar

To evaluate the improvement of FinePar on edge side query efficiency, we conduct end-to-end evaluation under varying amounts of query workloads, comparing it with four baseline methods. All queries are initiated from the edge. In this experiment, we selected three types of workloads and set the number of queries for each workload to range from 20 to 1000. Based on this setup, we obtained the following experimental results:

Efficiency of partition: The run time of the tested methods is shown in Fig. 6. The run time reduction (%) of FinePar over baseline methods is shown in Table III. We have the following observations:

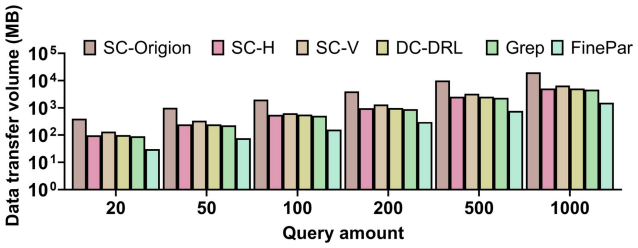
First, across all three datasets and varying query set sizes, FinePar consistently achieves the lowest end-to-end query runtime compared to the four baseline methods. On the TPC-DS benchmark, FinePar reduces the overall query runtime by approximately 40% , indicating its effectiveness in handling complex analytical workloads with diverse query patterns. The improvement is even more pronounced on TPC-H and JOB, where FinePar achieves up to 90% runtime reduction compared with the baselines. This demonstrates the effectiveness of FinePar's fine-grained partitioning strategy based on DRL, combined with a heat-aware allocation mechanism. By proactively placing frequently accessed partition blocks on the edge, FinePar significantly reduces the need for remote data transfers, thereby improving query responsiveness.

Second, as the number of queries increases, the overall runtime of all methods rises steadily, but FinePar shows a more stable and moderate growth trend. This indicates that FinePar is more adaptive to workload increases, maintaining low query latency even under high load. In contrast, baseline methods, particularly **SC-Origin**, **SC-H**, and **SC-V**, suffer from limited adaptability to dynamic workloads or depend solely on coarse-grained partitioning, thereby restricting edge-side optimization and resulting in significantly increased query times. Similarly,

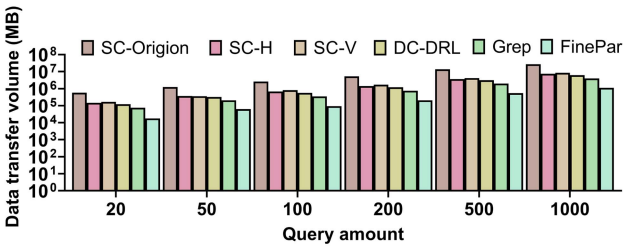
Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
DATA TRANSFER VOLUME REDUCTION (%) OF FinePar OVER BASELINE METHODS

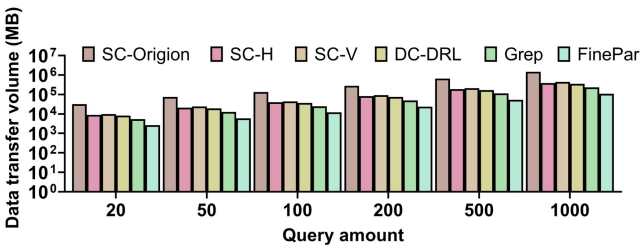
Dataset	TPC-DS					TPC-H					JOB				
Query amount	SC-Origion	SC-H	SC-V	DC-DRL	Grep	SC-Origion	SC-H	SC-V	DC-DRL	Grep	SC-Origion	SC-H	SC-V	DC-DRL	Grep
20	92.44%	69.19%	77.28%	69.38%	66.31%	96.91%	87.98%	89.31%	85.33%	76.82%	91.78%	70.80%	73.05%	67.81%	51.72%
50	92.39%	68.97%	77.31%	69.15%	66.06%	94.93%	83.15%	82.51%	80.72%	69.53%	92.03%	71.72%	75.65%	68.92%	53.38%
100	92.13%	71.77%	75.11%	71.90%	69.09%	96.40%	86.11%	88.71%	83.30%	73.62%	91.06%	69.99%	72.66%	67.38%	51.07%
200	92.46%	69.06%	77.38%	69.24%	66.17%	96.16%	85.64%	88.04%	82.95%	73.06%	91.42%	70.76%	74.02%	68.05%	52.07%
500	92.35%	69.88%	76.74%	70.05%	67.05%	95.91%	85.03%	86.88%	82.30%	72.04%	91.91%	71.35%	75.02%	68.44%	52.66%
1000	92.34%	70.02%	76.65%	70.19%	67.21%	95.78%	84.71%	86.61%	81.98%	71.53%	92.50%	71.60%	75.34%	68.71%	53.06%
Average	**92.35%**	**69.81%**	**76.75%**	**69.98%**	**66.98%**	**96.02%**	**85.44%**	**87.01%**	**82.76%**	**72.77%**	**91.78%**	**71.04%**	**74.29%**	**68.22%**	**52.33%**



(a) TPC-DS



(b) TPC-H



(c) JOB

Fig. 7. Comparison of data transfer volume: FinePar vs. Baselines.

DC-DRL and **Grep** also show less pronounced improvements, as both are limited to horizontal partitioning. Without the ability to further refine partitions along multiple dimensions, their optimization space is inherently narrower, which makes them less effective than FinePar in handling heterogeneous and rapidly changing query workloads.

Data transfer volume: The data transfer volume of the tested methods is shown in Fig. 7. The data transfer volume reduction (%) of FinePar over baseline methods is shown in Table IV. We have the following observations:

First, FinePar always shows lower total data transfer volume than the four baseline methods across all three datasets and different query set sizes. On the TPC-DS and JOB benchmarks, FinePar reduces data transfer by approximately 70%, demonstrating its ability to minimize communication overhead in complex analytical workloads. The improvement is even more substantial on the TPC-H benchmark, where FinePar achieves about 85% reduction in data transfer. It uses fine-grained partitioning and stores frequently accessed data at the edge. This reduces the amount of data transferred across layers.

Second, as the number of queries increases, the data transfer volume of FinePar grows more slowly. This indicates that FinePar not only lowers data transfer costs but also provides more stable performance under increasing workload demands.

In summary, the experimental results demonstrate that FinePar not only significantly reduces query latency and data transfer volume through intelligent partitioning but also scales more effectively with growing query complexity, making it particularly suitable for deployment in edge-centric CED architectures.

TABLE V
TRAINING TIME FOR TRAINING HYBRID PARTITION ADVISOR

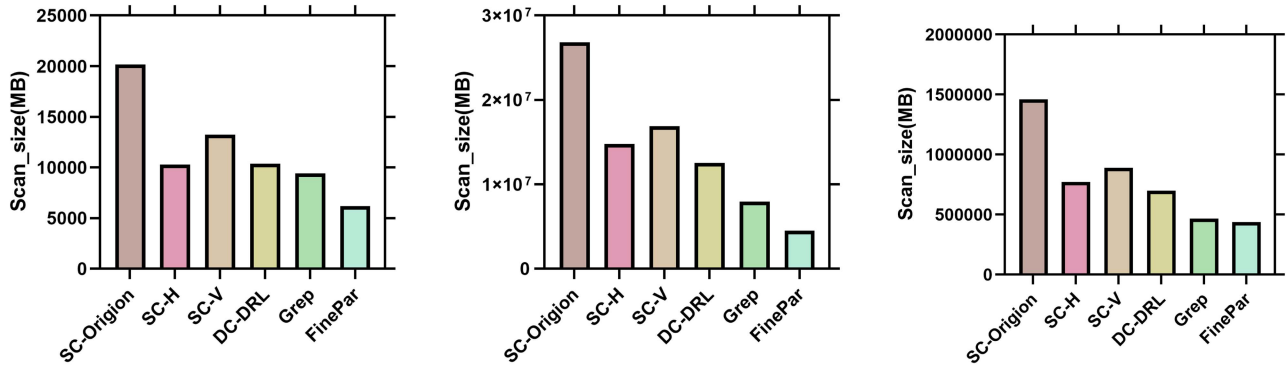
Datasets	TPC-DS	TPC-H	JOB
Train Time	62.02 min	15.84 min	54.56 min

C. Evaluation of Hybrid Partition Advisor

To evaluate the hybrid partitioning advisor in FinePar, we conduct comparative experiments against five representative coarse-grained baseline methods. These include **SC-Origin**, **SC-H**, **SC-V**, **DC-DRL**, and **Grep**. All methods are tested under identical workload conditions across three real-world datasets, ensuring a fair and consistent comparison. The training time is shown in Table V.

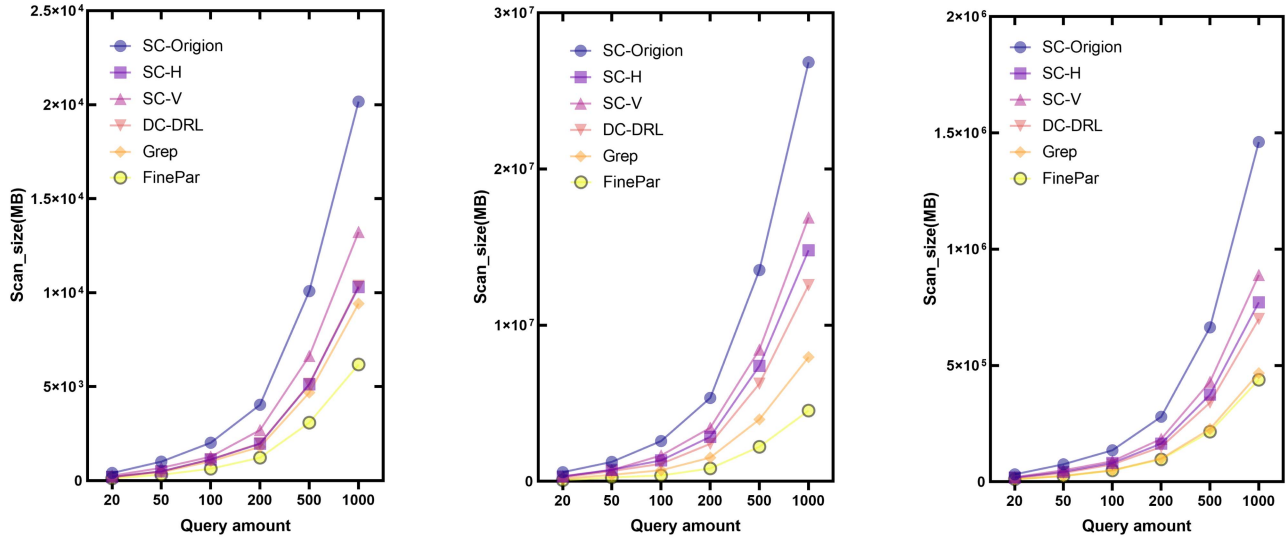
As illustrated in Fig. 8, FinePar consistently achieves the lowest scan size across all datasets and workload scenarios, showing stable and reliable performance in different experimental settings. This result clearly demonstrates that FinePar can generate fine-grained and workload-aware partitions, which allow the system to read only the necessary portions of data during query processing. It effectively reduces redundant data scanning and improves overall query efficiency. The static baselines, **SC-H** and **SC-V**, perform significantly worse because they rely on fixed and coarse-grained partitioning rules that cannot flexibly adjust to changing workload conditions. These methods make partitioning decisions in advance without considering runtime feedback or access pattern variations, which limits their

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.



(a) TPC-DS (b) TPC-H (c) JOB

Fig. 8. Hybrid partition advisor in FinePar vs. Baselines.



(a) TPC-DS (b) TPC-H (c) JOB

Fig. 9. Scalability on workload.

adaptability when workloads shift. Consequently, they suffer from high scan costs due to poor alignment with access patterns.

Although **DC-DRL** outperforms the static heuristics by leveraging learned workload patterns and adapting its partitioning decisions through reinforcement learning and **Grep** leverages graph learning to optimize data distribution across nodes, they still exhibit inherent limitations. Specifically, **DC-DRL** and **Grep** focus primarily on single-dimensional partitioning and lacks the ability to jointly consider horizontal and vertical dimensions within a unified optimization framework. In contrast, FinePar exploits hybrid partitioning to better capture multi-dimensional data access characteristics. Importantly, these results also validate the core capability of FinePar's partitioning advisor alone, even without employing the storage allocation strategy in CED. This underscores the intrinsic efficiency of FinePar in reducing I/O overhead and improving scan performance by optimizing partitioning structure. Overall, FinePar achieves superior performance by jointly considering granularity, workload alignment, and hybrid partition design.

D. Evaluation on Scalability

To validate the scalability of FinePar, we compared its performance with baselines under varying workload and data updates. In this experiment, we conduct experiments on three different datasets to evaluate the scalability of FinePar under increasing workload complexity, each with a range of query set sizes from 20 to 1000. To further assess FinePar's ability to handle growing data volumes, we maintain a workload of 20 queries and vary the size of the TPC-H dataset. Based on this setup, we obtained the following experimental results:

Scalability on Workload: Fig. 9 shows that FinePar consistently achieves a lower scan size than all baselines, regardless of the dataset or workload scale. This indicates its superior ability to generate partitioning schemes that align more precisely with access patterns, even as the query workload becomes more demanding.

More importantly, the growth rate of the scan size in FinePar is significantly slower than that of the other methods. While

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

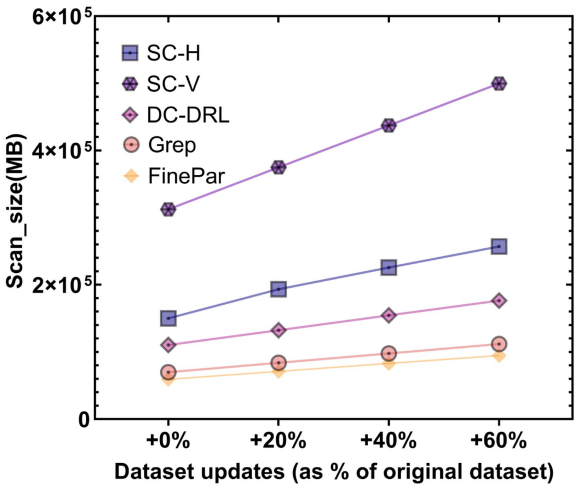


Fig. 10. Scalability on data.

baseline methods exhibit a near-linear or super-linear increase in scan cost as query volume scales, FinePar maintains a much flatter trajectory. This suggests that FinePar is capable of preserving storage and transmission efficiency even under high query loads, thanks to its fine-grained and adaptive partitioning strategy.

The static methods (**SC-H** and **SC-V**) struggle to respond to workload changes, resulting in substantial data overhead as query volume grows. **DC-DRL** and **Grep**, although dynamic, lacks hybrid partitioning and thus cannot fully capture evolving access patterns across dimensions.

Overall, these results demonstrate that FinePar exhibits strong scalability in query processing, maintaining low scan overhead while adapting to increased workload complexity. This makes it well suitable for real-world scenarios where query intensity and access patterns vary significantly over time.

Scalability on Data: Following the previous setting, we use the TPC-H dataset and train the RL advisor on the full database (100%). Subsequently, we expand the TPC-H data scale, incorporating up to 60% of new data to evaluate the model's adaptability to data growth. In Fig. 10, the original dataset size (+0%) is 30 GB, +20% means that the dataset grows to 36 GB. FinePar consistently achieves the lowest scan size across all dataset sizes, reaffirming its ability to minimize unnecessary data access through workload-aware, fine-grained partitioning. Notably, as the dataset size increases, FinePar's scan size grows at the slowest rate, resulting in the flattest growth curve among all compared methods.

This behavior underscores FinePar's superior scalability: it can effectively maintain partitioning quality and scan efficiency even when dealing with significantly larger datasets. This is critical for CED environments, where storage, computation, and bandwidth resources are limited, and excessive scan costs can quickly degrade system responsiveness.

In contrast, the static partitioning methods (**SC-H** and **SC-V**) exhibit steep growth in scan size as data volume increases, due to their inability to adapt partitioning granularity or layout to the changing data distribution. **DC-DRL** and **Grep**, while dynamic, lacks hybrid partitioning capability and shows less stability under large-scale data.

Overall, these results demonstrate that FinePar maintains strong partitioning performance and system efficiency even at scale, making it well-suited for modern data-intensive applications where data volumes continue to grow rapidly.

In summary, the results confirm that FinePar not only scales well with workload size but also with data size, making it suitable for real-world applications in CED architectures where both query volume and data volume can vary significantly.

VI. Related Work

A. Data Partitioning

Data partitioning is a fundamental technique in databases, which aiming to improve query efficiency and system scalability. Existing partitioning methods can be categorized into static partitioning and dynamic partitioning.

Static partitioning: Static partitioning methods typically determine partitioning rules during system initialization or data import and keep them fixed over time. Range partition and hash partition horizontally partitions tuples by applying a range or hash function to specific attributes. They are frequently used in database systems MySQL [9], Hive [10] and Spark [11]. These approaches do not depend on data distribution and require the DBAs to manually set specific attributes. It calls for automated method of recommending data partition. Fine-grained methods like Jigsaw [38] perform well for fixed workloads but are inefficient for dynamic ones due to high repartitioning costs. Unlike existing static or coarse-grained dynamic approaches, FinePar can adaptively recommend hybrid partitioning strategies based on both data and workload features.

Dynamic partitioning: There are some partition methods which generate horizontal or vertical partition strategies based on indexes, e.g., B-tree [13], R-tree [15], Quad-tree [16], Interval-tree [14], k-d tree [12], which is according to data distribution. For example, SpatialHadoop [49] partitions data according to spatial locality with a R-tree like mechanism. However, query workload distribution is not taken into account. Thus these methods overlook query workload distribution. Thus, they less suitable for dynamic CED environments. Other dynamic methods, such as heuristic and DRL based approaches. AutoPart [20] uses a bottom-up approach to receive a representative workload as input and utilizes categorical and vertical partitioning as well as selective column replication to recursively merge small partitions with similar access patterns into larger partitions. Sun et al. [18] propose a bottom-up clustering method that repeatedly combines unique binary feature vectors to obtain the lowest penalty. However, they tend to make simplifying assumptions, and it is notoriously hard to extend them to consider more complex workload. Hilprecht et al. [21] provide a learned partition advisor based on DRL for OLAP-style workloads. However, they only support horizontal hash partition and may have poor performance when the query workloads involve many dimensions [45], [46]. Our approach supports hybrid partition based on DRL, which can effectively support more query workloads.

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

B. Joint Query Execution in CED

With the development of CED architectures, several emerging database systems have begun to explore joint query execution across distributed layers. For example, the time-series database InfluxDB supports collaborative data management across cloud, edge, and terminal nodes, yet its performance in executing cross-layer queries remains limited. Apache IoTDB also provides CED data management capabilities [35], but its query optimization relies mainly on rule-based techniques and performs separate query execution for cloud and edge [36], which restricts true collaborative query processing and optimization.

Overall, most existing systems that support IoT or CED queries focus on query-level coordination, typically covering only one or two layers, and seldom address data placement and transfer efficiency as a core design concern. In contrast, FinePar tackles this challenge from a data distribution perspective, proposing a fine-grained partitioning strategy that minimizes data movement and enhances query performance across CED layers.

Remark: This paper is an extended version of our earlier work, WorthyPar [42], which introduced a DRL-based hybrid partitioning method for OLAP workloads. In this paper, we extend that line of research to address the unique challenges of CED architectures. The main contributions are as follows: (1) We propose FinePar, a fine-grained dynamic data partitioning framework tailored to the heterogeneous and resource-constrained nature of CED systems. (2) We formally define and prove the NP-hardness of the hybrid data partitioning problem in CED for the first time. (3) We further design an efficient partition allocation strategy that dynamically places partitions across cloud, edge, and device layers based on data temperature and real-time resource availability, enabling end-to-end system optimization. To the best of our knowledge, FinePar is the first framework that jointly considers fine-grained partitioning and adaptive placement to meet real-time processing demands in CED environments.

VII. Conclusion

In this paper, we propose FinePar, a fine-grained dynamic data partitioning framework based on DRL, coupled with an efficient data allocation strategy. In hybrid partition advisor training stage, we propose a DRL-based method to train an automatic hybrid partitioning advisor. This advisor recommends hybrid partitioning strategy based on data and workload of edge devices, dynamically optimizing the data partition across CED. In partition allocation in CED stage, we formulate a temperature function to estimate the each fine-grained partition temperature and employ a storage allocation strategy to optimize data placement across CED. It can further improve resource utilization and system performance in real-time. Extensive experimental results show that FinePar can reduce the edge side latency by 80% under resource constraints and dynamically adjust the partitioning strategy based on the workload. In future work, we plan to explore adaptive partition reallocation strategy that dynamically adjusts data placement across CED in response to changing workloads and resource fluctuations.

References

[1] M. S. Hossain and G. Muhammad, "Cloud-assisted Industrial Internet of Things (IIoT) – Enabled framework for health monitoring," *Comput. Netw.*, vol. 101, pp. 192–202, 2016.

[2] S. S. Cui, X. Wu, H. Z. Wang, and H. Wu, "Multi-modal data modeling technology and its application for cloud-edge-device collaboration," *Int. J. Softw. Inform.*, vol. 14, pp. 1154–1172, 2024.

[3] H. Liu, F. Han, and Y. Zhang, "Malicious traffic detection for cloud-edge-end networks: A deep learning approach," *Comput. Commun.*, vol. 215, pp. 150–156, 2024.

[4] S. S. Cui, H. Z. Wang, X. L. Liu, and X. O. Ding, "TempSched: A temperature-aware storage scheduler for time series across cloud-edge-device," in *Proc. Int. Conf. Data Eng.*, 2025, pp. 946–958.

[5] P. J. Lui, C. P. Li, and H. Chen, "Survey of intelligent partition and layout technology in database system," *Ruan Jian Xue Bao/J. Softw.*, vol. 39, pp. 3819–3843, 2022.

[6] H. Liu et al., "Survey of intelligent partition and layout technology in database system," *Ruan Jian Xue Bao/J. Softw.*, vol. 33, no. 10, pp. 3819–3843, 2022.

[7] M. Laszlo and S. Mukherjee, "Minimum spanning tree partitioning algorithm for microaggregation," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 07, pp. 902–911, Jul. 2005.

[8] B. Qarabaqi and M. Riedewald, "Merlin: Exploratory analysis with imprecise queries," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 2, pp. 342–355, Feb. 2016.

[9] MySQL, 1995. [Online]. Available: https://dev.mysql.com/doc/mysqlpartitioningexcerpt/8.0/en/partitioning-subpartitions.html

[10] Hive, 2007. [Online]. Available: https://hive.apache.org/

[11] "Spark," 2009. [Online]. Available: https://spark.apache.org/docs/latest/sqldatasources-parquet.html

[12] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Commun. ACM*, vol. 18, pp. 509–517, 1975.

[13] D. Comer, "The ubiquitous B-tree," *ACM Comput. Surv.*, vol. 11, no. 2, 1979, pp. 121–137.

[14] M. d. Berg, O. Cheong, M. v. Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*. Berlin, Germany: Springer, 2008.

[15] A. Guttman, "R-trees: A dynamic index structure for spatial searching," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1984, pp. 47–57.

[16] H. Samet, *Foundations of Multidimensional and Metric Data Structures*. San Mateo, CA, USA: Morgan Kaufmann Publishers Inc., 2006.

[17] C. Lin, M. Chen, and C. Lee, "insProgressive partition miner: An efficient algorithm for mining general temporal association rules," *IEEE Trans. Knowl. Data Eng.*, vol. 15, no. 04, pp. 1004–1017, Jul./Aug. 2003.

[18] L. Sun et al., "Fine-grained partitioning for aggressive data skipping," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 1115–1126.

[19] I. Alagiannis, S. Idreos, and A. Ailamaki, "H2O: A hands-free adaptive store," in *Proc. 2014 ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 1103–1114.

[20] S. Papadomanolakis and A. Ailamaki, "Autopart: Automating schema design for large scientific databases using data partitioning," in *Proc. Int. Work. Conf. Sci. Statist. Database Manage.*, 2004, pp. 383–392.

[21] G. C. Durand, R. Piriyev, M. Pinnecke, D. Broneske, B. Gurumurthy, and G. Saake, "Automated vertical partitioning with deep reinforcement learning," *Commun. Comput. Inf. Sci.*, vol. 1064, pp. 126–134, 2019.

[22] Z. Wu et al., "DRLPG: Reinforced opponent-aware order pricing for hub mobility services," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 6, pp. 3298–3311, Jun. 2025.

[23] X. H. Zhou, G. L. Li, J. H. Feng, L. Y. Liu, and W. Guo, "Grep: A graph learning based database partitioning system," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 94:1–94:24, 2023.

[24] L. Pan, Q. Cai, Z. Fang, P. Tang, and L. Huang, "A deep reinforcement learning framework for rebalancing dockless bike sharing systems," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 1393–1400.

[25] D. Seo, K. Chang, T. Cheong, and J. Baek, "A reinforcement learning approach to distribution-free capacity allocation for sea cargo revenue management," *Inf. Sci.*, vol. 571, pp. 623–648, 2021.

[26] Y. Tong, D. Shi, Y. Xu, W. Lv, Z. Qin, and X. Tang, "Combinatorial optimization meets reinforcement learning: Effective taxi order dispatching at large-scale," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 9812–9823, Oct. 2023.

[27] S. He and K. G. Shin, "Spatio-temporal capsule-based reinforcement learning for mobility-on-demand coordination," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 3, pp. 1446–1461, Mar. 2022.

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.

[28] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1582–1591.
[29] H. V. Hasselt, "Double Q-learning," in *Proc. Neural Inf. Process. Syst.*, 2010, pp. 2613–2621.
[30] Z. Wang, K. Fu, and J. Ye, "Learning to estimate the travel time," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 858–866.
[31] H. Chen et al., "InBEDE: Integrating contextual bandit with TD learning for joint pricing and dispatch of ride-hailing platforms," in *Proc. IEEE Int. Conf. Data Mining*, 2019, pp. 61–70.
[32] S. S. Eshkevari et al., "Reinforcement learning in the wild: Scalable RL dispatching algorithm deployed in ridehailing marketplace," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 3838–3848.
[33] X. Bai et al., "A cloud-edge-device collaborative offloading scheme with heterogeneous tasks and its performance evaluation," *Front. Inform. Technol. Electron. Eng.*, vol. 25, pp. 664–684, 2024.
[34] B. Hilprecht, C. Binnig, and U. Rohm, "Towards learning a partitioning advisor with deep reinforcement learning," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2019, Art. no. 6.
[35] B. McBride and D. Reynolds, "Survey of time series database technology," Tech. Rep., U.K. Centre for Ecology & Hydrology, 2020. [Online]. Available: https://nora.nerc.ac.uk/id/eprint/527832/
[36] P. Chen, W. He, W. Ma, X. Huang, and C. Wang, "IoTDQ: An industrial IoT data analysis library for apache IoTDB," *Big Data Mining Analytics*, vol. 7, no. 1, pp. 29–41, 2023.
[37] B. Hilprecht, C. Binnig, and U. Rohm, "Learning a partitioning advisor for cloud databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 143–157.
[38] D. Kang, R. Jiang, and S. Blanas, "Jigsaw: A data storage and query processing engine for irregular table partitioning," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2021, pp. 898–911.
[39] TPC-DS benchmark, 2011. [Online]. Available: http://www.tpc.org/tpcds/
[40] M. Poess and C. Floyd, "New TPC benchmarks for decision support and web commerce," in *Proc. Int. Conf. Manage. Data*, 2000, pp. 64–71.
[41] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really," *Proc. VLDB Endowment*, vol. 9, pp. 204–215, 2015.
[42] S. Cui et al., "WorthyPar: A workload-aware data hybrid partitioning advisor with deep reinforcement learning," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, 2025.
[43] M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher Eds. New York, NY, USA: Plenum Press, 1972.
[44] A. Levitin, *Introduction to the Design Analysis of Algorithms*. Reading, MA, USA: Addison-Wesley, 2003.
[45] Z. Li, M. L. Yiu, and T. N. Chan, "PAW: Data partitioning meets workload variance," in *Proc. IEEE Int. Conf. Data Eng.*, 2022, pp. 123–135.
[46] X. Qi, Z. Zhang, C. Jin, and A. Zhou, "BFT-Store: Storage partition for permissioned blockchain via erasure coding," in *Proc. IEEE Int. Conf. Data Eng.*, 2020, pp. 1926–1929.
[47] K. P. Bennett, M. C. Ferris, and Y. E. Ioannidis, "A genetic algorithm for database query optimization," Dept. Comput. Sci., Univ. Wisconsin-Madison, Madison, WI, USA, 1991.
[48] M. Irodova and R. H. Sloan, "Reinforcement learning and function approximation," in *Proc. FLAIRS Conf.*, 2005, pp. 455–460.
[49] A. Eldawy and M. F. Mokbel, "Spatialhadoop: A mapreduce framework for spatial data," in *Proc. IEEE Int. Conf. Data Eng.*, 2015, pp. 1352–1363.



Shuangshuang Cui (Student Member, IEEE) received the master's degree in computer science from the Harbin Institute of Technology, Harbin, China, in 2022. She is currently working toward the PhD degree in computer science with the Massive Data Computing Center, Harbin Institute of Technology, Harbin, China. Her research interests include cloud-edge-device collaborative database and AI for DB. Her publications include more than 10 papers in journals and conferences such as ICDE, DASFAA, etc.



Hongzhi Wang (Senior Member, IEEE) is a professor and Ph.D. supervisor with the Harbin Institute of Technology. Research Fields include Big Data management and analysis, and data quality. He has been PI for more than 10 projects including NSFC key project and National Technical support project. He has published more than 200 papers including VLDB, SIGMOD, and 6 books. He serves as the reviewer of more than 20 international journals, including *IEEE Transactions on Knowledge and Data Engineering* and PC member of more than 30 internal conferences.



Jinghan Lin received the bachelor's degree in computer science from the Harbin Institute of Technology, Harbin, China, in 2024. She is currently working toward the master's degree in computer science from the Harbin Institute of Technology, Harbin, China. Her research interests include AI for DB and database storage.



Xiaoou Ding received the PhD degree from the Harbin Institute of Technology, in 2021. She is an associated professor with the School of Computer Science and Technology, Harbin Institute of Technology, China. Her research work covers data cleaning and integration, time series data quality management. She has published more than 30 academic papers in international conferences and journals in database community, including *IEEE Transactions on Knowledge and Data Engineering*, VLDB, ICDE, CIKM. She serves as one of the proceeding chairs for VLDB 2024 and 2025, and as a member of PC for ICDE 2025.



Donghua Yang received the BS, MS and PhD degrees from the School of Computer Science and Technology, Harbin Institute of Technology, Harbin, China, in 1999, 2003, and 2008, respectively. He is currently an associate professor with the School of Computer Science and Technology, Harbin Institute of Technology. His research interests include Big Data management and cloud computing.

Authorized licensed use limited to: Harbin Institute of Technology. Downloaded on July 30,2026 at 10:34:03 UTC from IEEE Xplore. Restrictions apply.